

ECE391: Computer Systems Engineering

Machine Problem 2

Fall 2025
Checkpoint 1: Fri Sep 26 18:00
Checkpoint 2: Fri Oct 3 18:00
Checkpoint 3: Fri Oct 10 18:00

Drivers and Threading

Note: Must commit to the **mp2** folder inside the **main** branch in your provided GitHub repository by assignment deadline.

Make sure you merge any distributed code into the **main** branch.

Please read the entire document before you begin.

Introduction

It is good for software to be machine-independent. This means a layer is needed to translate independent abstractions into machine code. A program like Star Trek (1971), from your MP0, may use a familiar function like `printf` to print to the screen. But under the hood, `printf` will somehow need to communicate with the UART (Universal Asynchronous Receiver/Transmitter) to send a character to the screen. This communication occurs through a common medium that both the CPU and the device can access.

In this MP, we use **Memory-mapped I/O (MMIO)** for such communication. There are different methods to allow the CPU to interact with devices via MMIO. The simplest method is called **polling**, where the CPU repeatedly checks the device for updates by reading specific memory locations. However, polling can be inefficient if the CPU checks too frequently compared to the rate of I/O operations.

To address this, instead of having the CPU constantly check for updates, we can let the device notify the CPU when updates occur. This is done through a mechanism called **interrupts**. To manage interrupts, we use a device known as the **PLIC (Platform-Level Interrupt Controller)**, which coordinates different devices and manages interrupt status.

ECE391 MP2

Checkpoint 1	2
Real-Time Clock	2
PLIC	3
UART	4
Checkpoint 2	6
VirtIO Entropy Device	6
Checkpoint 3	7
RISC-V Timer	7
Thread Abstraction	10
Rubric and Grading	14
Getting Started	14

Checkpoint 1

Checkpoint 1 will be split into 3 tasks: RTC, PLIC, UART. This checkpoint focuses on writing your first device drivers and interrupt support. First, you will implement the RTC driver, a read-only driver to read the current time and date. Next, you will implement the PLIC, an interrupt-based MMIO communication between CPU and devices. Finally, you will implement a UART driver, an interrupt-based driver for serial communication. By the end, you will have the *Star Trek* game running on a system with your PLIC and UART drivers.

We recommend you read the following files before you begin:

- `device.c`
- `device.h`
- `devimpl.h`
- `dev/rtc.c`
- `dev/rtc.h`
- `plic.c`
- `plic.h`
- `dev/uart.c`
- `dev/uart.h`

Real-Time Clock

The Real-Time Clock (RTC) maintains the current time and date, even when the system is off. It timestamps events, schedules tasks, and provides time-based operations in embedded systems. In RISC-V systems, the RTC uses memory-mapped I/O (MMIO) and uses specialized registers. Two 32-bit registers, `time_low` and `time_high`, form a 64-bit timestamp. To ensure accuracy, read `time_low` first to latch the `time_high` value properly.

You'll receive detailed instructions for this part during your **discussion section**. The files that you will be editing are:

- `dev/rtc.c`

where you find comments saying “// `FIXME`”. You may not modify the `rtc_regs` and `rtc_device` structures.

RTC Function Implementations

These functions will implement the RTC device and its system interactions:

- `void rtc_attach(void* mmio_base)`
Attach RTC device
 This function will register the device with the system, initialize the serial device interface, and its memory-mapped registers. The given `mmio_base` is a pointer to the base address of the memory-mapped RTC registers.
- `int rtc_recv(struct serial* ser, void* buf, unsigned int bufsz)`
Read the current time
 This function gets the current real-time clock value and copies it to the given buffer. `ser` is a pointer to the device serial structure. You can use the `offsetof` macro to calculate the base address of the `rtc_device` structure from the value of `ser`. `buf` is a pointer to the buffer where the timestamp will be written to. `bufsz` is the size of the buffer in bytes. This function returns the number of bytes written to `buf`.
- `uint64_t read_real_time(volatile struct rtc_regs* regs)`
Get time from registers
 You will write this helper function that reads and returns the full 64-bit timestamp. `regs` is a pointer to the memory-mapped RTC registers.

PLIC

The PLIC priority-orders and distributes global interrupts in RISC-V systems. It connects interrupt sources (like I/O devices) to targets (such as hardware thread (hart) contexts). The PLIC consists of:

- Interrupt gateways (one per source)
- A controller that prioritizes and routes interrupts

Be sure to read the PLIC documentation.

For this checkpoint, you'll initialize the PLIC and enable interrupts for the UART.

The files that you will be editing are

- `plic.c`

where you find comments saying “// `FIXME`”. You may not modify the `plic_regs` structure.

PLIC Function Implementations

You are required to implement the following PLIC functions:

- `static inline void plic_set_source_priority(uint_fast32_t srcno, uint_fast32_t level)`
Set the priority level of an interrupt source
 This function changes the `priority` array. Each array entry matches an interrupt source.
- `static inline int plic_source_pending(uint_fast32_t srcno)`
Check if an interrupt source is pending
 This function returns 1 for a pending interrupt, 0 otherwise. This is decided by checking the bit in the `pending` array that corresponds to `srcno`.
- `static inline void plic_enable_source_for_context(uint_fast32_t ctxno, uint_fast32_t srcno)`
Enables an interrupt source for a context
 This function sets the appropriate bit in the `enable` array. It calculates the index based on `srcno` and `ctxno`, and sets the corresponding bit.
- `static inline void plic_disable_source_for_context(uint_fast32_t ctxno, uint_fast32_t srcid)`
Disables an interrupt source for a context
 This function clears the appropriate bit in the `enable` array. Like `plic_enable_source_for_context`, it must clear the correct bit for the given context and source.
- `static inline void plic_set_context_threshold(uint_fast32_t ctxno, uint_fast32_t level)`
Set the interrupt priority threshold for a context
 This function sets a threshold. Afterwards, the context should ignore interrupts below this priority.
- `static inline uint_fast32_t plic_claim_context_interrupt(uint_fast32_t ctxno)`
Claim an interrupt for a given context
 This function reads from the `claim` register and returns the interrupt ID of the highest-priority pending interrupt. It returns 0 if no interrupts are pending.

- `static inline void plic_complete_context_interrupt(uint_fast32_t ctxno, uint_fast32_t srcno)`
Complete the handling of an interrupt for a given context
 This function writes the interrupt source number back to the `claim` register, notifying the PLIC that the interrupt has been serviced.
- `static void plic_enable_all_sources_for_context(uint_fast32_t ctxno)`
Enable all interrupt sources for a given context
 This function sets all bits in the corresponding entry of `enable` array for the specified context.
- `static void plic_disable_all_sources_for_context(uint_fast32_t ctxno)`
Disable all interrupt sources for a given context
 This function clears all bits in the corresponding entry of `enable` array for the specified context.

UART

You have learned in class that the UART (Universal Asynchronous Receiver/Transmitter) protocol allows byte-by-byte CPU communication on a serial line. For this checkpoint, you will integrate UART communication for the MP0 Star Trek game. Write C functions to handle serial communication between Star Trek and the QEMU Emulator.

The files that you will be editing are

- `dev/uart.c`

where you find comments saying “// FIXME”. You may not modify the `uart_regs`, `ringbuf`, and `uart_serial` structures.

UART Function Implementations

You are required to implement the following UART functions:

- `int uart_serial_open(struct serial* ser)`
Prepare for UART communication
 The function opens an attached uart device. The `ser` parameter refers to the UART device serial structure that has to be opened. This function initializes the receive and transmit buffers, clears stale data by reading from RBR, enables the data ready interrupt, and registers the UART’s interrupt handle. You must also indicate the corresponding `uart_serial` is opened using the `opened` flag.
 This function returns 0 on success. If the given `ser` is already open, then return `-EBUSY`.
- `void uart_serial_close(struct serial* ser)`
Close UART communication
 This function shuts down UART communication. It is called indirectly by the `serial_close` function of the UART’s serial interface (set up in `attach_uart`). This function should unregister the device as needed, and prevent future interrupts by disabling all interrupts from device. Finally, it indicates the corresponding `uart_serial` is closed. If the given `ser` is not opened, then this function does nothing.
- `int uart_serial_recv(struct serial* ser, void* buf, unsigned int bufsz)`
Read from UART
 This function reads/receives data from the UART receive ring buffer. The number of bytes read will be between 1 and `bufsz` bytes, inclusive. It is called indirectly by the `recv` function of the UART’s serial interface (set up in `attach_uart`).
 This function returns the number of bytes received. If the given `ser` is not opened, then return `-EINVAL`.
Implementation: If the requested buffer size is valid, the function should copy characters from the receive ring buffer into the provided buffer. If the receive ring buffer is empty, it should wait until data becomes available. For CP1 and CP2, the function will “spin-wait”. However, for CP3, the function must wait on a condition variable. This function should also enable the corresponding interrupt to receive incoming data.

- `int uart_serial_send(struct serial* ser, const void* buf, unsigned int bufsz)`
Write to UART

This function writes/sends data to the UART transmit buffer. It will continually write characters to the transmit ring buffer from the provided buffer. It is called indirectly by the send function of the UART's serial interface (set up in `attach_uart`).

This function returns the number of bytes sent. If the given `ser` is not opened, then return `-EINVAL`.

Implementation: If the length of data is not zero, the function should begin writing to the transmit ring buffer. If the transmit ring buffer is full, the function should wait until there is space to write data; for CP1 and CP2, the function will use spinwaiting but for CP3 the function must wait on a condition. This function should also enable the corresponding interrupt to notify the UART that new data is ready to be sent.

- `void uart_isr(int srcno, void * aux)`
Service UART interrupt

This function serves as the UART interrupt service routine (ISR). It handles device interrupts (which mostly correspond to incoming and outgoing data events). Hardware triggers the ISR when UART events occur—it is not called directly by a program.

Implementation: This function manages UART interrupts by:

1. Checking LSR for UART state.
2. If data is available and receive buffer isn't full:
 - Store data from RBR to receive buffer.
3. If transmit buffer isn't empty:
 - Write from transmit buffer to THR.
4. Disable interrupts if:
 - (a) Receive buffer is full.
 - (b) Transmit buffer is empty.
5. For CP3, broadcast when:
 - (a) Receive buffer isn't empty.
 - (b) Transmit buffer isn't full.

Checkpoint 2

In each of the following files, you will add your implementation where you find comments labeled “// FIXME”.

- `viornng.c`

VirtIO Entropy Device

An operating system in general must communicate with external devices. For this course, we will use the VirtIO framework. For an introduction to the VirtIO device driver interface, you will need to implement a VirtIO Entropy (random number generation) device driver.

You must finish the implementation based on the VirtIO documentation. These documents are linked on the course website. You may find it especially helpful to read sections 2.1–2.7, 3.1, 4.2, and 5.4. Skip anything related to a “legacy” interface.

You are required to implement the following functions and structs in `dev/viornng.c`:

- `struct viornng_serial`
Define the necessary items required to implement the VirtIO entropy device, including `avail`, `used` virtqueues, and descriptors. Refer to the UART serial structure for other possibly important fields.
- `void viornng_attach(volatile struct virtio_mmio_regs* regs, int irqno)`
Attach RNG device
This function initializes the VirtIO Entropy device with the necessary IO operation functions and sets the required feature bits (if any). Also fills out the descriptors in the `virtqueue` struct. It attaches the `virtq_avail` and `virtq_used` structs using the `virtio_attach_virtq` function. Next, the serial struct is initialized with the correct interface. Finally, the device is registered.
- `int viornng_serial_open(struct serial* ser)`
This function makes the `virtq_avail` and `virtq_used` queues available for use. (Hint, read `virtio.h`)
Enables the interrupt source for the device, with the correct ISR.
This function returns 0 on success. If the given `ser` is already opened, then return `-EBUSY`.
- `void viornng_serial_close(struct serial* ser)`
This function resets the `virtq_avail` and `virtq_used` queues and prevents further interrupts. If the given `ser` is not opened, then this function does nothing.
- `int viornng_serial_recv(struct serial* ser, void * buf, unsigned int bufsz)`
This function reads up to `bufsz` bytes from the VirtIO Entropy device and writes them to `buf`. This is achieved by setting the appropriate registers to request entropy from the device, waiting until the randomness has been placed into a buffer, and then writing that data out to `buf`. Similar to the UART device, for CP2 this function can spin-wait on the data. However, for CP3 it must use a condition variable.
Returns the number of bytes of randomness successfully obtained. If the given `ser` is not opened, then return `-EINVAL`.
- `void viornng_isr(int irqno, void* aux)`
This function sets the appropriate device registers and wakes the thread up after waiting for the device to finish servicing a request. For CP3, it must broadcast a condition variable.

Checkpoint 3

In each of the following files, you will add your implementation where you find comments labeled “// FIXME”.

- `timer.c`
- `thread.c`
- `thrasm.s`
- `dev/uart.c` (Must modify for CP3)
- `dev/viorng.c` (Must modify for CP3)

RISC-V Timer

RISC-V uses the `mtime` and `mtimecmp` registers for timer interrupts. The system works as follows:

- `mtimecmp` is set to define the threshold.
- `mtime` is incremented every tick (about 0.1 microseconds on the QEMU RISC-V system).
- When $mtime \geq mtimecmp$, a timer interrupt occurs.
- If enabled, execution jumps to the interrupt service routine.

Your task (loosely speaking):

1. Implement functions to place a thread on a wait list, to be “awoken” by the timer.
2. Create an ISR to handle timer interrupts and wake threads after their sleep duration.

Timer API

The timer interface uses two main data structures:

- `struct alarm`
 1. `cond`: Condition variable for waiting threads.
 2. `tawake`: Time (in ticks) of alarm initialization, reset, or wake-up.
 3. Pointer to next alarm struct.
- `struct alarm * sleep_list`
 1. Linked list of alarm objects.
 2. Sorted in increasing `tawake` order.
 3. Stores alarms waiting for interrupts.

When a program needs to interact with the timer, it creates an alarm object. This gets added to `sleep_list` with the appropriate `tawake`.

When the timer interrupt service routine fires, any expired alarms are removed, and their condition variables are broadcast. This system helps efficiently manages timed events and thread synchronization. Below is the structure definition of the alarm object:

```

struct alarm {
    /* Condition variable for waiting for timer duration */
    struct condition cond;
    /* Next alarm in linked list */
    struct alarm * next;
    /* # of ticks where alarm was reset/initialized/should wake */
    unsigned long long twake;
};

```

Any programs that interact with the timer will do so through the following functions:

- `void alarm_init(struct alarm * al, const char * name)`
Initialize the alarm
 This function should perform necessary initialization for an alarm struct just declared. If no name is given (i.e., `name == NUL`), give your alarm a sensible name (e.g., “alarm”).
- `void alarm_sleep(struct alarm * al, unsigned long long tcnt)`
Put the alarm to sleep
 This function adds the alarm to `sleep_list`, setting the thread to sleep for `tcnt` ticks relative to most that alarm’s `twake` time. Described in more detail below.
- `void alarm_reset(struct alarm * al)`
Reset alarm object
 This function resets an alarm object so that `twake` is aligned with current time in ticks.

Timer Function Implementations

We provide you with two helper functions:

- `void set_stcmp(uint64_t stcmp_value)`
Sets mtimecmp to a specific value
 Argument `stcmp_value` is the new value to write into the `mtimecmp` register. Since `mtimecmp` cannot be set directly from supervisor mode, the function temporarily transfers control to machine mode in order to set the register to the desired value. Note that if the desired number of ticks is already passed, which occurs when `mtime` is greater than the `stcmp_value`, an interrupt will be fired after the function completes and `mtimecmp` is set.
- `unsigned long long rdttime(void)`
Returns the current value stored in the mtime register

You are required to implement the following timer functions in `timer.c`:

- `void alarm_init(struct alarm * al, const char * name)`
Initialize alarm fields
 Argument `name`, which may be null, is an optional name that can be passed to the condition variable. The `al` argument is a pointer to the alarm object to initialize the members of.
Implementation: The initialization of the condition variable `cond` and the alarm object pointer should be straightforward. However, make sure that `twake` is initialized to the proper tick count. Remember, the `twake` field is supposed to keep track of the most recent time in ticks between an alarm’s initialization, reset, and wake-up events.
- `void alarm_sleep(struct alarm * al, unsigned long long tcnt)`
Have the current thread sleep
 Argument `al` is a pointer to the alarm object that the thread will wait for a wake-up event on. Argument `tcnt` is the number of ticks to put the thread to sleep for *relative to the most recent initialization, reset, or wake-up*

event for the alarm object. The thread is to be put to sleep until the appropriate time in ticks at which the sleep duration ends.

Implementation:

1. Check if sleep duration has passed.
 - If so, return immediately.
 2. Otherwise,
 - Update `twake` for next wake-up, if applicable.
 - Add alarm to `sleep_list`, in proper position.
 - Put current thread to sleep.
 3. Update `mtimecmp` register as needed.
 - Use the `rdtime` function (backed by pseudoinstruction).
 - Use `set_stcmp` to set the interrupt expiry time.
 4. Enable timer interrupts when adding new alarms (why?).
- `void handle_timer_interrupt(void)`
Service timer interrupts
 This function is an interrupt service routine. It should handle timer interrupts by waking all ready threads and updating the `sleep_list` linked list and `mtimecmp` registers accordingly.
Implementation: Your implementation should make use of the `rdtime` function to remove all alarms that are past their threshold wake-up event time from the `sleep_list`, and wake all threads waiting on said alarm conditions.
 Afterwards, you should use the `set_stcmp` function to appropriately set the timer interrupt threshold to waiting for the next wake-up event on the `sleep_list`, if there is one. If the `sleep_list` is empty, disable timer interrupts.

Thread Abstraction

A thread is lightweight execution unit, as discussed in class. In modern Linux kernels, the scheduler is a core mechanism that manages thread execution. In this context, we are most interested in the fact that it must maintain queues of threads in various states. Schedulers will be discussed in more detail later, we promise!

For this checkpoint, we implement a cooperative multitasking approach. Co-operative threads voluntarily give up control when they are waiting for any arbitrary reason. Such times include when waiting on device I/O, a timer interrupt, user input, or another similar event that runs slowly relative to the CPU. During this time, control can be transferred to another available thread.

For our purposes there are 5 possible thread states:

- **UNINITIALIZED:** For a very brief period during its creation, a thread is in the UNINITIALIZED state, before transitioning to the READY state.
- **READY:** A thread in the READY state is ready to run, but is not currently running. All threads in the READY state must be on the ready-to-run list ready list. The ready-to-run list must contain only threads in the READY state.
- **RUNNING:** A thread that is currently being executed by the CPU is in the RUNNING state. A thread that is RUNNING must not be on the ready-to-run list ready list.
- **WAITING:** A thread waiting on a condition variable is in the WAITING state. When the condition is signaled, the thread transitions to the READY state. A thread can only enter the WAITING state by calling condition wait function.
- **EXITED:** When a thread exits, it enters the EXITED state. Such a thread is not removed from the thread table until its parent calls thread join or thread join all.

The frequent context switches between threads create the illusion of each thread executing on its own CPU (with a common memory). Notice that this is different from a time slice. Food for thought: in our system, what can we say about when and how often context switches happen?

Thread API

Each thread is represented with a struct. Here is its declaration, with some descriptive comments:

```
struct thread {
    /* must be first member (thrasms.s) */
    struct thread_context ctx;
    /* index into thrtab */
    int id;
    /* current state of thread */
    enum thread_state state;
    /* thread name */
    const char * name;
    struct thread_stack_anchor * stack_anchor;
    void * stack_lowest;
    /* ptr to parent thread */
    struct thread * parent;
    /* ptr to next (linked) thread */
    struct thread * list_next;
    /* ptr to condition thread is waiting on */
    struct condition * wait_cond;
    /* ptr to condition variable for thread to signal parent(!) on exit */
    struct condition child_exit;
};
```

The process will contain 4 threads in total:

- **Main Thread:** This thread only spawns two other threads, the Star Trek thread and the Rule 30 thread. Afterwards, the thread will call thread join and wait for the Star Trek thread to terminate. When the Star Trek thread terminates, it will signal the main thread through a condition variable and the main thread will exit. This will shutdown the entire program, including the Rule 30 thread.
- **Idle Thread:** The idle thread exists to soak up any unused CPU cycles and to ensure that there is always a thread that is ready to run. It is not necessary to have an idle thread, but having one simplifies the scheduler. The idle thread is very simple: if there are no threads that are ready to run, it uses the RISC-V wait-for-interrupt instruction wfi to wait for an interrupt. On real hardware, this would stop the CPU clock to conserve power. On QEMU, this stops executing instructions until an interrupt occurs. *Check your understanding: Why is it correct to stop and wait for an interrupt when there are no READY threads?*
- **Star Trek:** The Star Trek game is the same as in Checkpoint 1.
- **Rule 30:** Rule 30 is a ASCII screensaver that consists of cells that are either in an off or on state. The state of every pixel in a line depends on its neighbors on the previous line. The thread will periodically be woken up by the timer using a condition variable and draw the next iteration of cells based on the previous iteration. Users of the thread API do not manipulate or refer to the thread structure shown above directly. Instead, they refer to threads by a thread identifier (TID).

Condition Variables

A condition variable is a synchronization primitive that (in our system) supports two operations:

- **Waiting** on a condition: Suspends the calling thread until another thread calls condition broadcast. Such a waiting thread is in the WAITING state. This function is not safe to call this from an interrupt context (why?).
- **Broadcasting** a condition: Signals (wakes up) all threads waiting on the condition. Signaling a thread puts it in the READY state. This function may be called from an interrupt context.

Broadcasting a condition variable is like posting on a message board—any thread waiting on your condition (“subscribed to your thread”) will be woken up (“get a notification”).

You are required to use condition variables with the UART and VirtIO Entropy device for CP3.

Some systems allow for a more granular wake-up, often called signaling, wherein a single thread waiting on a condition can be awakened directly. We will not use (or require) that behavior for this assignment.

Finally, here is the declaration for the condition variable struct, with some comments:

```
struct condition {
    /* Optional name for debugging*/
    const char * name;
    /* Threads waiting on this condition */
    struct thread_list wait_list;
};
```

Thread Function Implementations

You are required to implement the following thread functions in `thread.c`:

- `int spawn_thread(const char * name, void (*entry)(void), ...)`
Create and schedule a new thread
 The argument `name`, which may be null, is the name of the new thread. The `entry` argument is a function

pointer to the function that the thread will start executing. Up to eight additional arguments may follow the `entry` argument; these arguments will be passed to the entry function when it is executed. The created thread is scheduled for execution. The `thread_spawn` function does not cause a context switch; the newly created thread will be executed eventually after the calling thread yields to the scheduler. This function may *not* be called from an ISR. *Spot check: why not?*

Implementation: Your implementation should call `create_thread` to create the new thread. Note that the `create_thread` function may return null; you must return error code `-EMTHR`, which means that the maximum number of threads has been reached, and no new threads can be created. Your implementation must set the thread state to `THREAD_READY` and add the newly created thread to the ready list. Your implementation must ensure that when the new thread is scheduled to execute, it will jump to the `entry` function provided to `thread_spawn`, passing the additional arguments which were provided. See the POSIX documentation for the `stdarg.h` header to see how to handle such arguments.¹

To do this, you will need to initialize the `context` structure of the new thread in such a way that when `_thread_switch` is called to switch to the new thread, execution will start in `entry` as described above. Additionally, it must be arranged such that when `entry` finishes, `thread_exit` is called.

Advice: Remember that the beginning is a very delicate time. We suggest implementing and using the special `_thread_startup` helper function to start your new thread with the correct arguments. As a hint, we have included a code fragment in `thread_spawn` between `va_start` and `va_end` to extract the optional arguments to `thread_spawn` and save them in the part of the `thread_context` structure normally used to save registers `s0` through `s7`. However you decide to initialize your `thread_context` struct, you must also initialize the static `idle_thread` struct in the same manner.

- `void _thread_startup(void)`

Assembly helper function for thread spawn

Helper function for `thread_spawn`. It should assist in `thread_spawn`'s goal of ensuring that a newly created thread starts in the `entry` function with the proper arguments, and calls `thread_exit` when `entry` finishes.

Implementation: Implementation details are left as a decision for the reader to make. The only “hard requirement” is that your implementation should match with how thread context is expected to be in `thread_spawn`.

- `void running_thread_exit(void)`

Exit the current running thread

Terminates the currently running thread. This function does not return.

Implementation: Your implementation should first check if the currently running thread is the main thread. If so, `halt_success()` is called. Otherwise, it should set the current thread's state to `THREAD_EXITED`. Next, it should signal the parent thread in case it is waiting for the current thread to exit. Lastly, it should call `running_thread_suspend()`, which should not return (*why?*). If we somehow manage to return, call `halt_failure()`.

- `int thread_join(int tid)`

Wait for a child thread to exit

If `tid` is *not* zero, the function waits for the identified child of the running thread to exit. Otherwise, the function waits for *any* child of the running thread. This function returns the thread id of the child thread that exited, or an error. The `thread_join` function returns `-EINVAL` if `tid` is not zero and if the identified thread does not exist or is not a child of the running thread. The `thread_join` function returns `-EINVAL` if `tid` is zero and the running thread does not have any children.

Implementation: Your implementation must cover two cases:

1. Child already exited. If the child has already exited, `thread_join` does not need to wait. It should return immediately.
2. Child still running. If the child is still running, the parent should wait on the condition variable `child_exit` in its own struct `thread`. Note that an existing child signals this condition in `thread_exit`.

¹<https://pubs.opengroup.org/onlinepubs/009604599/basedefs/stdarg.h.html>

In either case, the parent should release the resources used by the child thread by calling `thread_reclaim`.

- `void condition_broadcast(struct condition * cond)`

Wake up threads waiting on the condition variable

This function may be called from an ISR. Calling `condition_broadcast` does *not* cause a context switch from the currently running thread. Waiting threads are added to the ready-to-run list in the order they were added to the wait queue.

Implementation: Waking up a thread entails:

1. Changing its state from `THREAD_WAITING` to `THREAD_READY`,
2. placing it on the ready list, and
3. removing it from the list of threads waiting on the condition.

After broadcasting the condition, the set of threads waiting on the condition should be empty.

- `void running_thread_suspend(void)`

Suspend current thread and switch to a ready one

Suspends the currently running thread and resumes the next thread on the ready-to-run list using `_thread_swch` (in `thrasm.s`). Must be called with interrupts enabled. Returns when the current thread is next scheduled for execution. If the current thread is `TP`, it is marked `THREAD_READY` and placed on the ready-to-run list. Note that `running_thread_suspend` will only return if the current thread becomes `READY`.

Implementation: Your implementation should be a simple round-robin scheduler. The thread being suspended, if it is still runnable, is inserted at the tail of ready list, and the next thread to run is taken from the head of the list. The calling thread may be `THREAD_RUNNING`, `THREAD_WAITING`, or `THREAD_EXITED`. Only in the first case should your self-suspend place the current thread back on the ready-to-run list (and update its state accordingly). The last thing this function should do is check if the thread that called it is in the `THREAD_EXITED` state. If so, clean up (free its stack).

Rubric and Grading

Rubric and grading policies for MP2 will be released to the course website.

Submitting the Assignment

Your MP2 code will be tested by an autograder at each checkpoint. This autograder will run against your code on the `main` branch at the deadline of each checkpoint. If you choose to work ahead (ie. working on CP2 before CP1 is due), which is encouraged, please keep your code on a separate branch. The easiest way to do this to use

```
git checkout -b <new_branch>
```

After the deadline of the checkpoint, you can merge your next checkpoint code into the `main` branch. This is due to potential differences in behavior expected in each checkpoint. We do not want to penalize you for working ahead.

There will be a nightly autograder that will begin a few days before the deadline of each checkpoint. This autograder **does not** test for the full expected functionality. The primary purpose is to test for compilation and base functionality. The nightly autograder will have no impact on your MP2 grade. Getting a full score on the nightly autograder **does not** mean you will get a full score on the functionality section of the MP.

Getting Started

To get started, pull from the release repository. The `mp2` folder will contain all the files you will need for the entire `mp2`.

You are not allowed to add or remove C and assembly files from the given code. You may only modify the following files:

- CP1:
 - `dev/rtc.c`: Do not modify the definition of the `rtc_regs` and `rtc_device` structures.
 - `plic.c`: Do not modify the definition of the `plic_regs` structure.
 - `dev/uart.c`: Do not modify the definition of the `uart_regs`, `ringbuf`, and `uart_serial` structures.
- CP2:
 - `dev/viorng.c`: You will need to implement the `viorng_serial` structure.
- CP3:
 - `thrasm.s`
 - `thread.c`: Do not modify the definition of the `thread_context`, `thread_stack_anchor`, `thread` structure and the `thread_state` enumeration.
 - `timer.c`

Modifying files other than the ones listed above may cause your code to not be compatible with the autograder.

Running and Testing

Once you have completed your implementation, you can run your code using QEMU, as with previous MPs:

```
# run normally
make run-cp12
```

```
# OR, run stopped (for GDB)
make debug-cp12
```

Corresponding commands exist for later checkpoints, too. Please read the `Makefile` to see the target names.

To view the output in a different terminal, use the screen command:

```
screen /dev/pts/N
```

where `N` is the number provided by QEMU on the first line of output. This will ensure you can see the UART output properly.

To help you verify your implementation, you are encouraged to write your own test cases.